

What is Swig Interface

The Swig interface is a project in the MLib Visual Studio Solution that generates MLib API to other programming languages other than C++. We currently produce interfaces for Java and C#. The MLib methods exposed via Swig solely come from the Vanilla Addin and the ETrading Addin projects.

How to build Swig interface

The Swig interface project is called **MLIB_CLIENT_API**. It's not part of the MLib Solution file normally. Developers will need to set up their desktop before they are able to build this project successfully. Add the MLIB_CLIENT_API project back to the MLib solution before carrying out the build.

How to expose a MLib method via Swig

The current MLIB_CLIENT_API is already exposing a large range of methods. Developers who wish to expose a new method via Swig will need to make the following changes:

1. Create a new file for the method to be exposed.

Take the existing file `mirGetBusinessDays.cpp` for example. This file only defines one function and that is 'mirGetBusinessDays'. This is the name of the API method exposed to other programming languages.

2. Define the function signature of your interface.

The interface function signature normally follows that of the corresponding method in the `validation_api` project. For instance, the signature of 'mirGetBusinessDays' fully resembles that of `validation_api::tryMirGetBusinessDays`.

3. Perform type marshalling on inputs

From the existing interface files, developers can probably quickly conclude that the main task of implementing a new interface file is to perform type marshalling on the inputs before calling the corresponding method in the `validation_api` project. The type marshalling is required because the API interfaces only support primitive C++ types as well as certain commonly used container types. If the corresponding `validation_api` methods is using any in-house types such as `MBString`, type marshalling is required here.

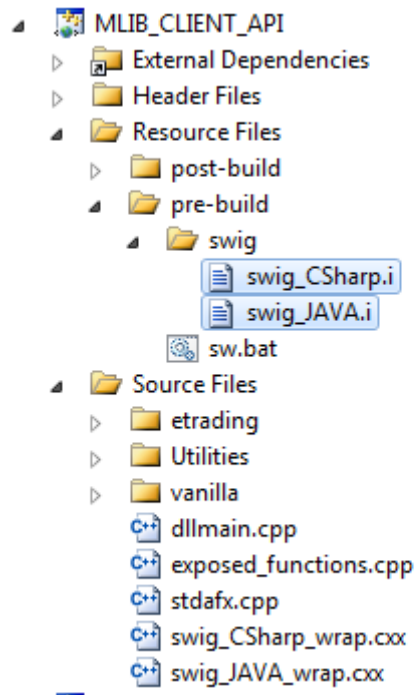
4. Don't forget to put a try-catch block around all the implementation code of each interface method.

The catch block is only required to catch `MError` type exceptions.

The background of this design is: at the `validation_api` level structured exception handling has been put in place. Any catchable structured exceptions thrown from within `validation_api` or any levels below it get caught inside `validation_api` which are then re-thrown to the Swig interface level as `MError` exception.

5. Add the new interface method to the Swig files

Swig files are files that inform Swig where to look for the interface files. We have one Swig file for each target programming language which means we have one for Java and one for C#. You can file them in:



There is already a long list of

#include "xxxx"

And

%include "xxxx"

Make sure you list your new file under both lists.

This is the last step. Build your MLIB_CLIENT_API to get the results.

Distributing MLib API (Swig Interfaces)

At the end of the build, post-build scripts are run to arrange the build result files properly into two groups, one for Java and one for C#. These results can be found under
\\ MLib\\mlib\\src\\MLIB_CLIENT_API.

API results are currently located under: **X:\\Secure\\DIR_MLib_API**